Diploma in Computer Science & Information Technology

SBTE, Bihar — Semester IV

Database Management System

Course Code: 2418403

Unit-wise Detailed Notes

UNIT 1: Overview of Database Management System

1.1 Database — Concept and Need

A database is an organized, structured collection of related data stored electronically so that it can be easily accessed, managed, and updated. Unlike file systems, databases reduce redundancy and inconsistency.

Need of Database: Large organizations handle vast amounts of data (student records, bank transactions, inventory). Databases provide efficient storage, retrieval, and management of this data.

Advantages of Database

- Reduced data redundancy — same data not stored multiple times.
- Data consistency — changes reflect everywhere automatically.
- Data integrity — constraints enforce correctness of data.
- Data security — access control mechanisms protect sensitive data.
- Concurrent access — multiple users can access data simultaneously.
- Data independence — application programs unaffected by data storage changes.

1.2 Database Management System (DBMS)

A DBMS is software that manages databases. It acts as an interface between users/applications and the physical database. Examples: MySQL, Oracle, PostgreSQL, MS SQL Server, MongoDB.

File Processing System vs DBMS

- File systems store data in separate files with no relationship management; DBMS stores interrelated data with relationships.
- File systems have high redundancy; DBMS minimizes redundancy.
- File systems lack concurrent access control; DBMS handles multiple users.
- File systems have no standardized query language; DBMS uses SQL.

1.3 Relational Data Model

In the relational model, data is organized in tables (relations). Key concepts: Domain = set of permissible values for an attribute. Attribute = column of a table. Tuple = row/record of a table. Relation = table with rows and columns.

1.4 Types of Database Systems

- **Centralized Database System:** All data stored at a single location/server. Simple management but single point of failure.
- **Distributed Database System:** Data distributed across multiple physically separate locations connected by a network. Improves availability and performance.
- **Parallel Database System:** Multiple processors work together to execute queries on a shared database. Improves query performance.
- **Client/Server Database System:** Database runs on a server; clients access it over a network. Most common architecture for modern applications.

UNIT 2: Relational Database Management System (RDBMS)

2.1 RDBMS Terminology

RDBMS is a DBMS based on the relational model. Data is stored in tables with relationships between them. An RDBMS enforces ACID properties (Atomicity, Consistency, Isolation, Durability) for transactions.

- **Schema:** The logical structure/design of a database — table names, column names, data types, constraints.
- **Instance:** The actual data stored in the database at a particular moment.

2.2 Entity-Relationship (E-R) Model

The E-R model is a conceptual data model used for database design. It uses graphical diagrams (E-R diagrams) to represent data entities and their relationships.

Components of E-R Diagram

- **Entity:** A real-world object or concept (e.g., Student, Course). Represented by a rectangle.
- **Attribute:** Property of an entity (e.g., StudentID, Name). Represented by an ellipse. Types: Simple, Composite, Multivalued, Derived, Key attribute.
- **Relationship:** Association between entities (e.g., Student ENROLLS IN Course). Represented by a diamond.
- **Strong Entity:** Has its own unique key attribute.
- **Weak Entity:** Cannot be uniquely identified by its own attributes alone; depends on a strong entity. Represented by double rectangle.

Relationship Types (Cardinality)

- **One-to-One (1:1):** One entity instance relates to at most one instance of another entity.
- **One-to-Many (1:N):** One entity instance relates to many instances of another entity. (e.g., one Teacher teaches many Students)
- **Many-to-Many (M:N):** Many instances of one entity relate to many instances of another. (e.g., Students enroll in multiple Courses)

2.3 Keys in DBMS

- **Super Key:** Any set of attributes that uniquely identifies a tuple. Can contain extra attributes.
- **Candidate Key:** Minimal super key — no proper subset of it is a super key. A table can have multiple candidate keys.
- **Primary Key:** The chosen candidate key used as the main identifier. Cannot be NULL. Each table has exactly one primary key.
- **Alternate Key:** A candidate key that is not chosen as the primary key.
- **Foreign Key:** An attribute in one table that references the primary key of another table. Establishes referential integrity between tables.

2.4 Integrity Constraints

- **Domain Constraint:** Value of an attribute must belong to its defined domain (data type and range).
- **Entity Integrity Constraint:** Primary key value cannot be NULL.

- **Referential Integrity Constraint:** Foreign key value must match an existing primary key value in the referenced table, or be NULL.
- **Key Constraint:** Primary key values must be unique.

2.5 Conversion of E-R Diagram to Tables

Rules: (1) Each strong entity becomes a table with its attributes as columns, primary key as table's primary key. (2) Each weak entity becomes a table; its primary key includes foreign key of owner entity. (3) 1:1 relationship — add foreign key to either side. (4) 1:N relationship — add foreign key to the "many" side. (5) M:N relationship — create a new table with primary keys of both entities as foreign keys.

UNIT 3: Relational Database Design

3.1 Functional Dependency

A functional dependency (FD) X -> Y means: for any two tuples with the same X values, they must have the same Y values. X determines Y, or Y is functionally dependent on X. Example: StudentID -> StudentName, StudentID -> Major.

Closure of FDs

The closure F+ of a set F of FDs is all FDs that can be derived from F using Armstrong's axioms: Reflexivity (if Y subset X then X->Y), Augmentation (if X->Y then XZ->YZ), Transitivity (if X->Y and Y->Z then X->Z).

3.2 Normalization

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. It involves decomposing tables based on functional dependencies.

First Normal Form (1NF)

A relation is in 1NF if: All attributes have atomic (indivisible) values. No repeating groups or multi-valued attributes. Each column has a unique name. Each row is unique (has a primary key). Example: A table with a "Phone Numbers" column containing multiple phone numbers violates 1NF.

Second Normal Form (2NF)

A relation is in 2NF if: It is in 1NF AND every non-key attribute is FULLY functionally dependent on the entire primary key (no partial dependencies). Only applies when the primary key is composite. Eliminate partial dependencies by moving them to a separate table.

Third Normal Form (3NF)

A relation is in 3NF if: It is in 2NF AND no non-key attribute is transitively dependent on the primary key. Transitive dependency: A -> B -> C where A is PK, B is non-key, C is non-key. Move transitive dependencies to a separate table.

Boyce-Codd Normal Form (BCNF)

A stronger version of 3NF. A relation is in BCNF if: For every FD X -> Y, X must be a super key. BCNF eliminates anomalies not handled by 3NF. Sometimes decomposing to BCNF loses some FDs.

3.3 Denormalization

Denormalization is the intentional introduction of redundancy to improve read performance. Used in data warehouses and reporting databases where query speed is more important than update efficiency. Benefits: faster query performance, fewer joins needed. Drawbacks: data redundancy, update anomalies.

UNIT 4: Relational Algebra and SQL

4.1 Relational Algebra

Relational algebra is a procedural query language that takes relations as input and returns a relation as output. It forms the theoretical foundation for SQL.

Fundamental Operations

- **Select (sigma):** Filters rows based on a condition. sigma condition(R). Equivalent to SQL WHERE clause.
- **Project (pi):** Selects specific columns. pi col1,col2(R). Equivalent to SQL SELECT columns.
- **Union (U):** Combines tuples from two compatible relations (same schema). Eliminates duplicates.
- **Set Difference (-):** Returns tuples in R1 but not in R2. R1 - R2.
- **Cartesian Product (x):** Combines every tuple of R1 with every tuple of R2.
- **Rename (rho):** Renames a relation or its attributes.

4.2 Join Operations

- **Natural Join:** Joins relations on all common attributes, eliminating duplicate columns.
- **Equi-Join:** Join based on equality condition between specified attributes.
- **Inner Join:** Returns matching rows from both tables based on join condition.
- **Left Outer Join:** Returns all rows from left table; matching rows from right; NULL for no match.
- **Right Outer Join:** Returns all rows from right table; NULL for non-matching left rows.
- **Full Outer Join:** Returns all rows from both tables; NULL where no match exists.

4.3 SQL — Structured Query Language

SQL is the standard language for relational database management. It is declarative — you specify WHAT data you want, not HOW to get it.

DDL — Data Definition Language

- CREATE TABLE: Creates a new table with specified columns and constraints.
- ALTER TABLE: Modifies structure of existing table (add/modify/drop columns, add constraints).
- DROP TABLE: Permanently deletes a table and its data.
- TRUNCATE TABLE: Removes all rows but keeps the table structure.

DML — Data Manipulation Language

- INSERT INTO: Adds new rows to a table.
- UPDATE: Modifies existing rows based on a condition.
- DELETE FROM: Removes rows based on a condition.

- SELECT: Retrieves data from one or more tables.

SQL Clauses

- WHERE: Filters rows based on condition.
- GROUP BY: Groups rows with same values for aggregate functions.
- HAVING: Filters groups (used with GROUP BY); like WHERE for groups.
- ORDER BY: Sorts result set (ASC or DESC).

Aggregate Functions

- COUNT(*) — counts number of rows. SUM(col) — sum of values. AVG(col) — average. MAX(col) — maximum. MIN(col) — minimum.

Nested and Correlated Queries

A subquery is a SELECT inside another SELECT. Correlated subquery references columns from the outer query and executes once per row of the outer query.

4.4 TCL — Transaction Control Language

- COMMIT: Permanently saves all changes made in the current transaction.
- ROLLBACK: Undoes all changes since the last COMMIT or SAVEPOINT.
- SAVEPOINT name: Creates a save point within a transaction for partial rollback.
- SET TRANSACTION: Sets transaction properties like isolation level.

UNIT 5: Other Schema Objects

5.1 Views

A view is a virtual table based on the result of a SELECT query. It does not store data physically — data is derived from the underlying base tables each time the view is accessed. Views simplify complex queries and provide data security by restricting access.

Creating and Using Views

CREATE VIEW view_name AS SELECT col1, col2 FROM table WHERE condition;

Querying a view: SELECT * FROM view_name; The view can be used like a regular table in queries.

Updatable Views: Views can sometimes be used for INSERT, UPDATE, DELETE if they are based on a single table with no GROUP BY, DISTINCT, or aggregate functions.

DROP VIEW view_name — permanently removes the view (base data is unaffected).

5.2 Sequences

A sequence is a database object that generates unique sequential numbers automatically. Commonly used to generate primary key values. CREATE SEQUENCE seq_name START WITH 1 INCREMENT BY 1 MAXVALUE 999999;

To use: INSERT INTO table VALUES(seq_name.NEXTVAL, ...); Current value: seq_name.CURRVAL.

5.3 Indexes

An index is a data structure that improves the speed of data retrieval operations on a database table. Similar to a book's index — allows finding data without scanning the entire table. Tradeoff: faster queries but slower inserts/updates.

Types of Indexes

- **Simple Index:** Index on a single column. CREATE INDEX idx_name ON table(column);
- **Unique Index:** Ensures all values in the indexed column are distinct. CREATE UNIQUE INDEX idx ON table(col);
- **Composite Index:** Index on multiple columns. Useful for queries filtering on multiple columns.

DROP INDEX idx_name — removes the index.

5.4 Synonyms

A synonym is an alternative name (alias) for a database object like a table, view, or sequence. Useful for hiding the actual object name or location. CREATE SYNONYM syn_name FOR schema.table_name;

Once created, the synonym can be used in SQL queries just like the original object name. DROP SYNONYM syn_name removes it.
